A45 Team ColumbiaGals

Project Title: PicoRV32 SoC Microcontroller

LAYOUT VIDEO REVIEW: (If you feel the video is too long, feel free to skim the slides instead - they are roughly the same!) https://youtu.be/_MC6p484aMs

Discord name	Affiliation	Experience	Role
farihah__ahmed	Columbia University	Undergraduate	Team lead, digital designer
rusmiya	Columbia University	Undergraduate	Digital designer
cowyamm	Columbia University	Undergraduate	Digital designer

Results - Signoff Table

Check	Tool	Result
DRC	Magic (full)	**4 → 0** — flow output had 4 M3.1 violations (Metal3 width < 0.56 µm); healed, full re-check clean
LVS	Netgen	Circuits match uniquely — 13,966 devices / 13,021 nets
Antenna	OpenROAD	PASS — 0 violations
Setup	OpenSTA, 9 corners	MET — worst +75.07 ns @ max_ss_125C_4v50
Hold		MET — worst +0.328 ns @ min_ff_n40C_5v50

⚠️ Note for reviewers: the submitted GDS (gds/chip_top_full.gds) is fully healed — DRC 0 as checked. If the flow is *re-run* from config.json, the known M3.1 tool issue (OpenLane #1549) regenerates 4 slivers; please apply gds/heal_metal3.tcl afterward (librelane → heal → re-verify), or a waiver for those 4 documented shapes would be needed.

Corners = {min,nom,max} RC extraction × {ff_n40C_5v50, tt_025C_5v00, ss_125C_4v50}. Max-cap/slew counts are checker reporting noise; detailed reports resolve to 'no violations found'

Corner/Group	Hold Worst Slack	Reg to Reg Paths	Hold TNS	Hold Vio Count	of which reg to reg	Setup Worst Slack	Reg to Reg Paths	Setup TNS	Setup Vio Count	of which reg to reg	Max Cap Violatio…	Max Slew Violati…
Overall	0.3275	0.3275	0.0000	0	0	75.0668	75.0668	0.0000	0	0	118	2638
nom_tt_025C_5v00	0.5391	0.5391	0.0000	0	0	87.0656	87.0656	0.0000	0	0	85	2181
nom_ss_125C_4v50	1.0194	1.0194	0.0000	0	0	76.6026	76.6026	0.0000	0	0	89	2186
nom_ff_n40C_5v50	0.3294	0.3294	0.0000	0	0	91.7556	91.7556	0.0000	0	0	78	2038
min_tt_025C_5v00	0.5371	0.5371	0.0000	0	0	87.8110	87.8110	0.0000	0	0	64	1782
min_ss_125C_4v50	1.0176	1.0176	0.0000	0	0	77.9148	77.9148	0.0000	0	0	68	1927
min_ff_n40C_5v50	0.3275	0.3275	0.0000	0	0	92.2339	92.2339	0.0000	0	0	55	1792
max_tt_025C_5v00	0.5416	0.5416	0.0000	0	0	86.1948	86.1948	0.0000	0	0	115	2589
max_ss_125C_4v50	1.0216	1.0216	0.0000	0	0	75.0668	75.0668	0.0000	0	0	118	2634
max_ff_n40C_5v50	0.3316	0.3316	0.0000	0	0	91.1967	91.1967	0.0000	0	0	109	2638

Final routed die — chip_top_full, 1000x1000 um, 1mm^2

DMEM -
64 words x 8 bits SRAM

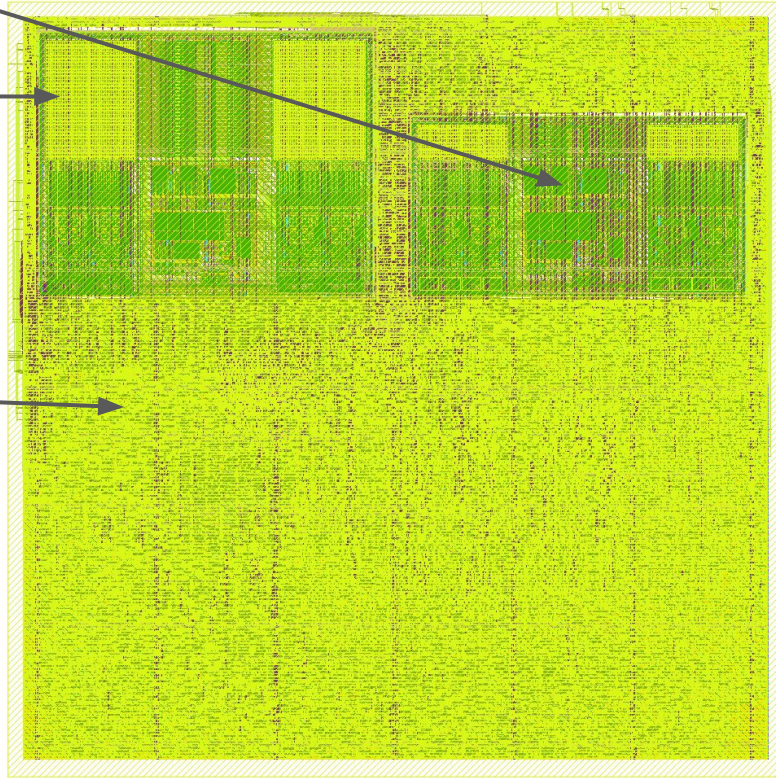
IMEM -
256 words x 8 bits SRAM

PicoRV32 RISC-V CPU core + bus + peripherals
(the yellow sea of gates you see) -
46,344 std cells

Metrics:

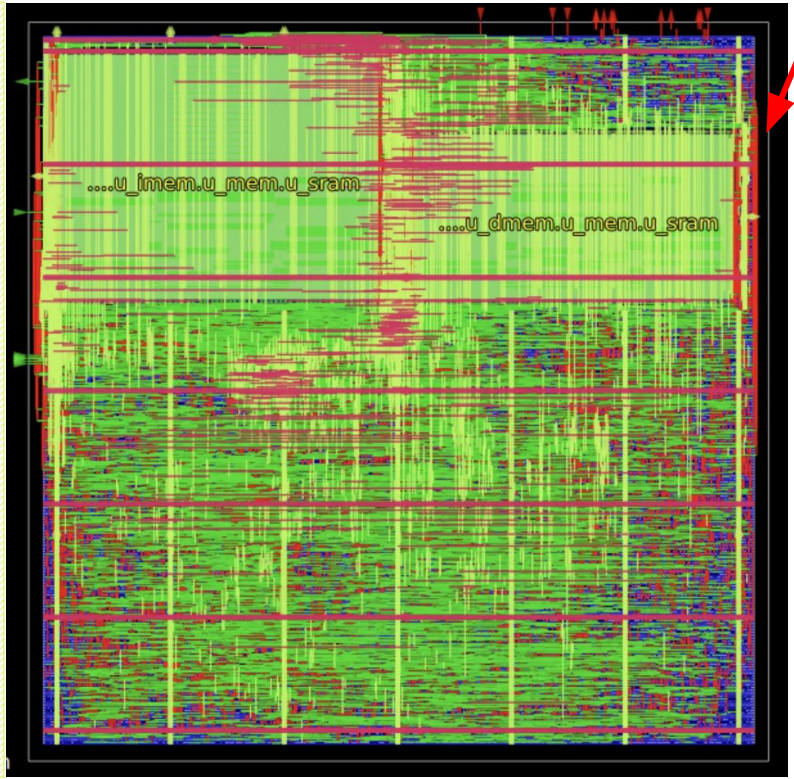
(1) 10 MHz
(2) DRC 0
(3) LVS Match
(4) Antenna Clean
(5) Timing met - all 9 corners
(4) 72.6% utilization

Left (KLayout — GDS view): Final GDS — includes every polygon, including fill cells and all metal layers. Rendered from gds/chip_top_full.gds (DRC-clean, LVS-verified).



Right (OpenROAD — design view): Same die from the place-and-route database — standard cells (green/blue), power straps (red), SRAM macros labeled. Fill omitted, making logic structure visible.

PDN: VDD/VSS strap grid
(red lines - vertical Metal straps and horizontal rails)



Challenges/Major Decisions: Ibex→ Pico Swap

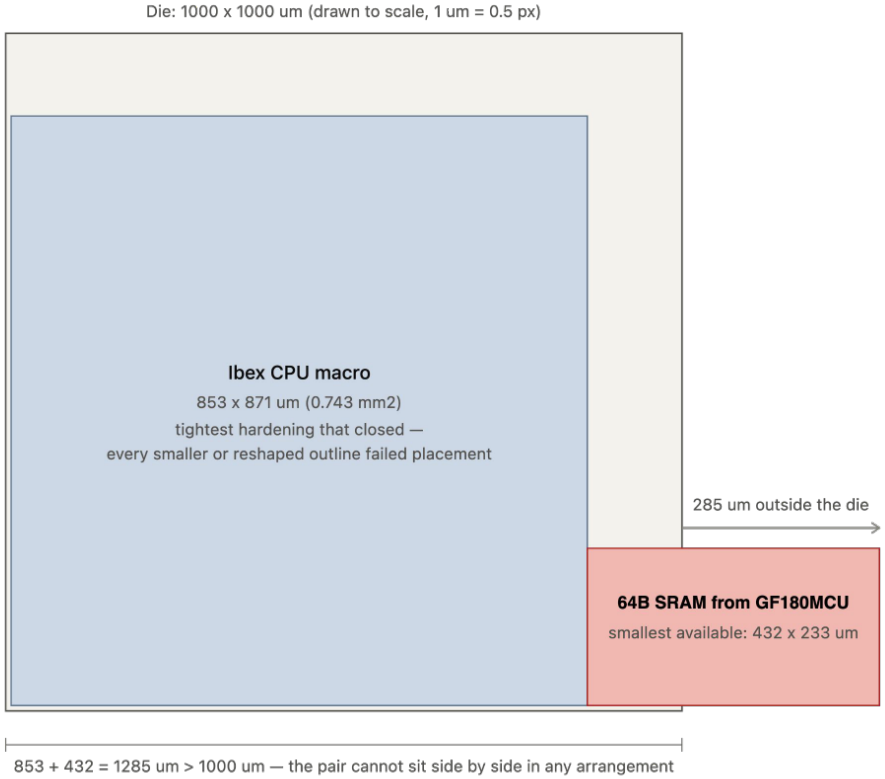
When picking an open-source core in the front-end design, we picked the Ibex Core from lowRISC, as it has seen multiple successful tape outs.

In Yosys synthesis (the last part of front end design), our area was measured to be 0.845mm^2 (**no wires, no gaps, no clock tree**).

However, when starting the back-end physical design, we soon realized when running the librelane flow that **after adding thousands of hold buffers, clock-tree buffers, diodes added during PnR,** the design no longer fit within 1mm^2! It grew to 3mm^2 with the extra items after the librelane flow ran fully.

The deeper problem was geometric: Ibex could only be hardened as a rigid 853 x 871 um macro (0.743mm^2 for the CPU alone). **Even the smallest SRAM macro (64B) is 431um x 232um, so there's no way a square orientation would fit (and we wanted 2 SRAM macros - for IMEM and DMEM).** See diagram →

Also, every time we tried to re-harden IBEX at other aspect ratios it failed placement. Forcing Ibex into any other outline left too little area or routing room inside the box, so its cells could no longer be legally placed and routed. **853 x 871um was the only shape that closed, which meant our SRAM macros would not fit unless we had a bigger die.**



Challenges/Major Decisions: Ibex→ Pico Swap

Our fix was to change CPUs: we replaced Ibex with PicoRV32, another silicon-proven open-source core. We originally picked Ibex for performance (2-stage pipeline), while PicoRV32 executes each instruction over 3–14 cycles — so we traded performance for far better area.

The real fix is integration style: PicoRV32 is small enough (one Verilog file, ~⅓ the gates) to synthesize inline at the top level, so it has **no shape at all** — no macro, no tiling problem; its gates flow around the SRAMs like any other logic. Ibex's denser logic required a rigid pre-hardened macro, and no shape of that macro tiled with the SRAMs inside 1 mm². Result: entire SoC logic = **0.335 mm² synthesized** — less than half of Ibex's macro alone — **placed and closed at exactly 1000×1000 µm**.

	Ibex (macro)	PicoRV32 (inline)
Synthesis estimate	0.845 mm² total	0.335 mm² SoC logic
CPU footprint	853×871 µm rigid block (0.743 mm²)	no block — distributed gates
After full PnR	~3 mm² effective — did not fit	**1.0 mm² die, closed** (72.6% util)
Tiling with 2 SRAMs	impossible (853+432 > 1000)	n/a — nothing to tile

Floorplan facts (explain 72.6% util)

Parameter	Value	Notes
Die / Core	1000×1000 µm / 960×960 µm	We reserve a 20 µm margin on each side of the core for the padring hookup
Macros	imem sram256×8 @ (40, 620) · dmem sram64×8 @ (520, 620), N orientation	We pin only the two SRAMs at fixed coordinates (lower-left corner, µm, die origin bottom-left); they are rigid blocks. **All other logic — CPU, bus, peripherals — is standard cells placed freely by the optimizer. Design decision due to area constraints - no major CPU block.**
Macro halo	4 µm	We keep a 4 µm band around each SRAM free of standard cells so the router has clean access to the macro pins.
Target density	0.6 (achieved 72.6% util)	We instruct the placer to fill regions to ~60% locally; after hold buffers, clock tree, and fill insertion, overall core utilization lands at 72.6%.
PDN	VDD/VSS strap grid, no core ring	We build the on-die power grid (vertical straps + horizontal rails, macros connected explicitly); the thick core ring and pad power will be added at the padring stage, so our grid only needs to be tappable — which a regular strap grid is.
Pins	20 signal, edge-placed	We place all 20 signal pins on the core boundary where the shuttle's padring connects; VDD/VSS come in through the power grid rather than signal pins.

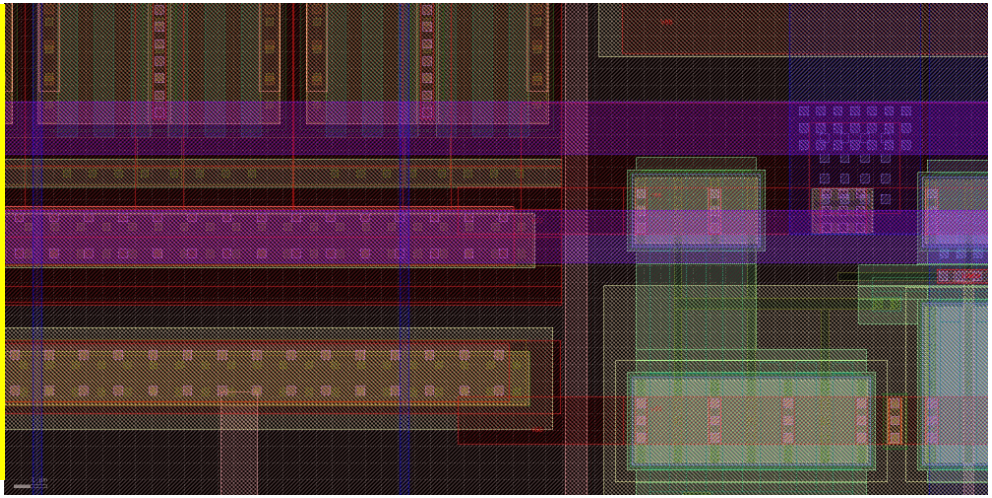
Metal3 Heal — the one DRC issue, found and fixed

The one DRC issue we found, understood, and fixed. The PDN's Metal3 connection to the GF180 SRAM macros leaves **4 sub-µm slivers** (rule M3.1: width < 0.56 µm) — a known tool issue (OpenLane #1549 / OpenROAD PR #2814) regenerated on every flow run.

Fix: `heal_metal3.tcl` (see github link) — a 14-line Magic script that paints the 4 flagged shapes to legal width. The DRC report's polygon coordinates match the script's paint boxes 1:1 — the fix targets exactly the flagged geometry, nothing else.

Note: reruns of the flow regenerate these 4 slivers — the submitted GDS is healed; if re-running from config, apply the script afterward (librelane → heal → re-verify) **(full DRC + LVS on healed GDS)** → 0 violations, circuits still match uniquely.

The heal region at y≈651 µm, drawn from the signoff GDS: the wide horizontal Metal3 power straps (purple) crossing the SRAM boundary — the slivers occurred where these straps met the macro's own metal, and were painted into the straps at the 4 flagged coordinates.



Code Blame 14 lines (14 loc) · 379 Bytes

```
1   drc off
2   gds read openlane/chip_top_full/runs/RUN_2026-08-08_06-31-45/final/gds/chip_top_full.gds
3   load chip_top_full
4   box 158.435um 650.80um 240.000um 651.36um
5   paint m3
6   box 240.000um 650.80um 246.985um 651.36um
7   paint m3
8   box 638.435um 650.80um 720.000um 651.36um
9   paint m3
10  box 720.000um 650.80um 726.985um 651.36um
11  paint m3
12  gds write /tmp/healed2.gds
13  puts "HEAL DONE"
14  quit -noprompt
```

Before/After “Heal” Script:

Before: 4 M3.1 violations at documented coordinates. After heal: DRC 0, LVS match.

BEFORE:

```
/foss/designs/ibex_soc > cat openlane/chip_top_full/runs/RUN_2026-08-08_06-31-45/65-magic-drc/reports/*.rpt | tail -10
Metal3 width < 56 (M3.1)
----------------------------------------
 158.435um 650.995um 240.000um 651.165um
 240.000um 650.995um 246.985um 651.165um
 638.435um 650.995um 720.000um 651.165um
 720.000um 650.995um 726.985um 651.165um
----------------------------------------
[INFO] COUNT: 4
[INFO] Should be divided by 3 or 4
```

AFTER:

After heal — DRC_RESULT.txt records the Magic full-check outcome on the healed GDS (0 violations); the LVS line is quoted directly from gds/lvs_report.rpt, netgen's own output ('Circuits match uniquely'). Both files are committed in the repo for independent inspection

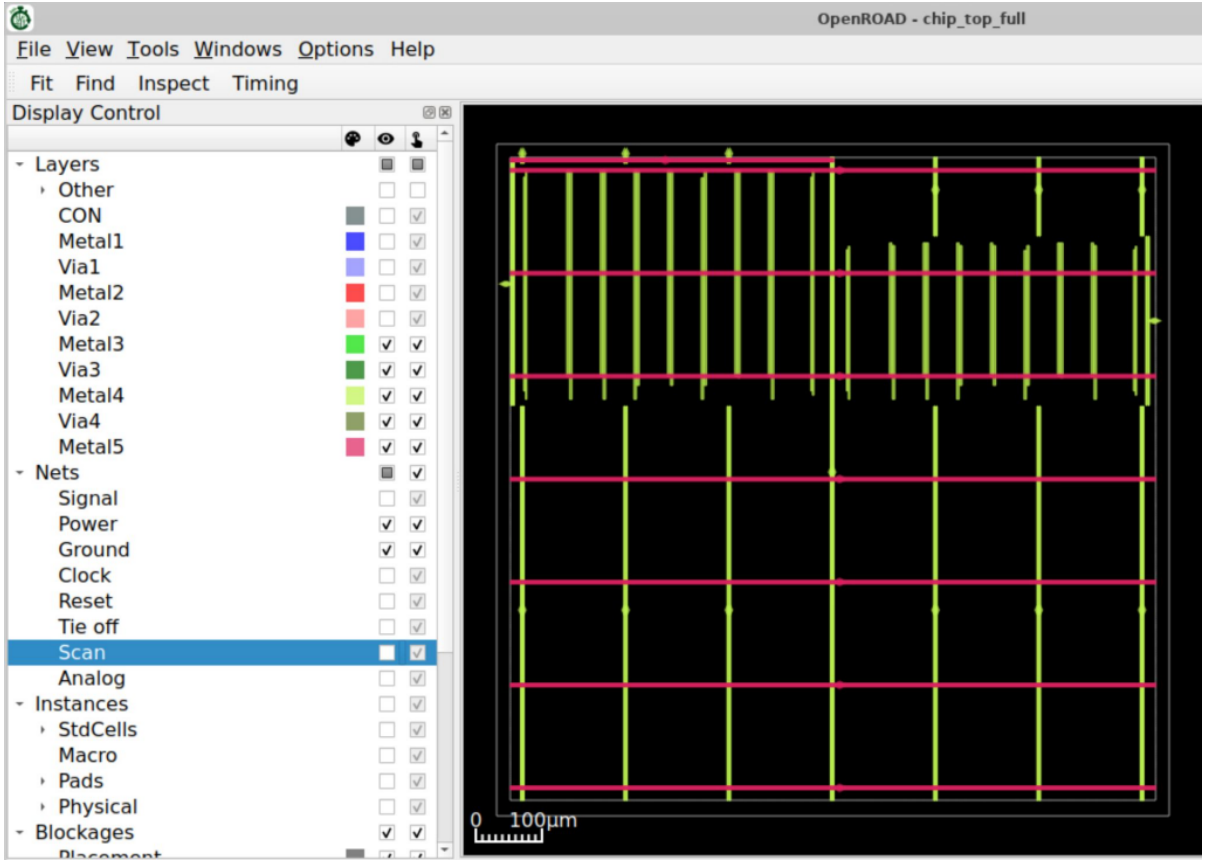
```
[/foss/designs/ibex_soc > echo "=== DRC (healed GDS) ===" && cat gds/DRC_RESULT.txt && echo && echo "=== LVS ===" && grep -A3 "Final result" g]
ds/lvs_report.rpt
=== DRC (healed GDS) ===
DRC: 0 (Magic full check on healed GDS, 2026-08-08)

=== LVS ===
Final result: Circuits match uniquely.
.
```

Power, Ground, and Current Paths

Image - OpenRoad with only the power grid visible: The power delivery network, isolated: vertical VDD/VSS straps (Metal4, green) × horizontal straps (Metal5, pink), forming the grid that feeds every cell row below. The dense strap groups at top are the explicit macro connections over both SRAMs. Per-row Metal1 rails (hidden here) hang beneath this grid.

Single 5V domain (VDD/VSS) — no domain crossings, no supply separation needed. Per-cell power via horizontal Metal1 rails in every row; bulk delivery via vertical + horizontal PDN straps on upper metals; both SRAM macros connected explicitly (PDN_MACRO_CONNECTIONS) with 10 µm halos. Well/substrate taps are built into every GF180 std cell and macro; spacing verified by DRC. Current demand is modest — 10 MHz, 180 nm, minimum-drive cells — so strap widths are generous relative to load: IR-drop and ground-bounce risk low. No core ring by design: the padring stage adds ring + pad power; our grid needs only to be tappable.



Analog Matching, Symmetry, and noise (from rubric)

Not applicable — fully digital design. No analog or mixed-signal circuitry anywhere: clock generation is a synthesizable divider (scan-programmable) with an external-clock fallback pin — no PLL, no oscillator, no matched pairs, no sensitive analog nodes. The only analog-adjacent structures are the SRAM macros' internal sense amps, which are pre-qualified foundry IP used as-is.

Reliability and Physical Design Risks

- Antenna: 0 violations — targeted diode insertion (threshold 200, input ports).
- Latch-up: well/substrate taps integrated in every std cell + macro (tap-included library); tap-spacing rules enforced by DRC — 0 errors.
- Voltage domains: single 5V domain — no cross-domain spacing issues; all devices are 5V-rated cells matching the supply.
- Electromigration / current density: negligible at 10 MHz with minimum-drive cells; no high-current signal nets; PDN straps sized far above demand.
- ESD: provided at the padring (pad-level clamps) — is this outside our block scope per Chipathon integration?
- The one 'DRC-clean but risky' item we found: the Metal3 slivers — caught, healed, re-verified (previous slide)."

Top Level Integration and Connectivity

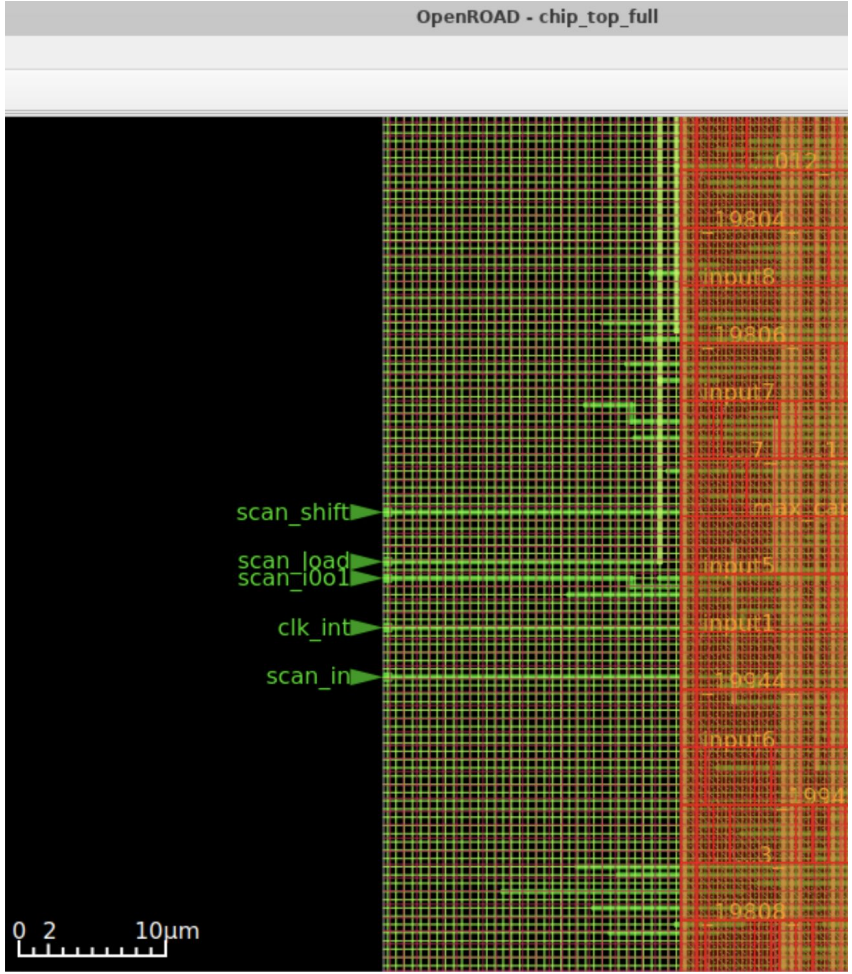


Image: Left die edge, zoomed: named signal pins (scan_shift, scan_load, scan_i0o1, clk_int, scan_in) placed on the core boundary, entering the routing grid toward the cell fabric. Pin names in the layout match the RTL ports one-to-one — proven end-to-end by LVS.

All 20 signal pins placed on the core boundary where the shuttle padring connects; **VDD/VSS enter via the power grid**, not signal pins.

Pin names identical across RTL ↔ netlist ↔ layout — top cell is `chip_top_full` everywhere, and **LVS matching uniquely proves name and connectivity agreement end-to-end**.

Clearances: **20 µm ring** around the core for padring routing, **4 µm macro halos** internally. Submission wiring: **info.yaml (root) → lvs_config.json → gds/chip_top_full.gds + chip_top_full.pnl.v — the LVS-verified pair**. Full pin table in PINOUT.md on Github.

Pins sit on standard routing layers at the boundary, so the padring's router can connect directly; no special interface cells required.

Questions for Reviewers

(1) Any general feedback on what we should improve before tapeout?

(2) For the 4 M3.1 violations that reruns regenerate: is our heal-script workflow (librelane → heal → re-verify) acceptable for the dry run, or should we file a formal waiver for those 4 documented shapes? **(if so, how do we formally submit a waiver? We just aren't too familiar with the process)**

(3) We recognize 10 MHz is conservative — with +75 ns of setup slack, is there value in retargeting a faster clock at this stage, or is stability the right priority for a first tapeout?

(a) If we target a faster clock, are there any design levers you suggest we pull?

(4) We'd like to grow the memories (currently 256 B imem + 64 B dmem), but larger macros push utilization past 72.6%. In your experience, how much headroom above ~73% is safe on GF180 before placement/routing risk outweighs the capacity gain?

We have some extra slides after this. Thank you!

Extra Slide: Heat Maps

Placement Density

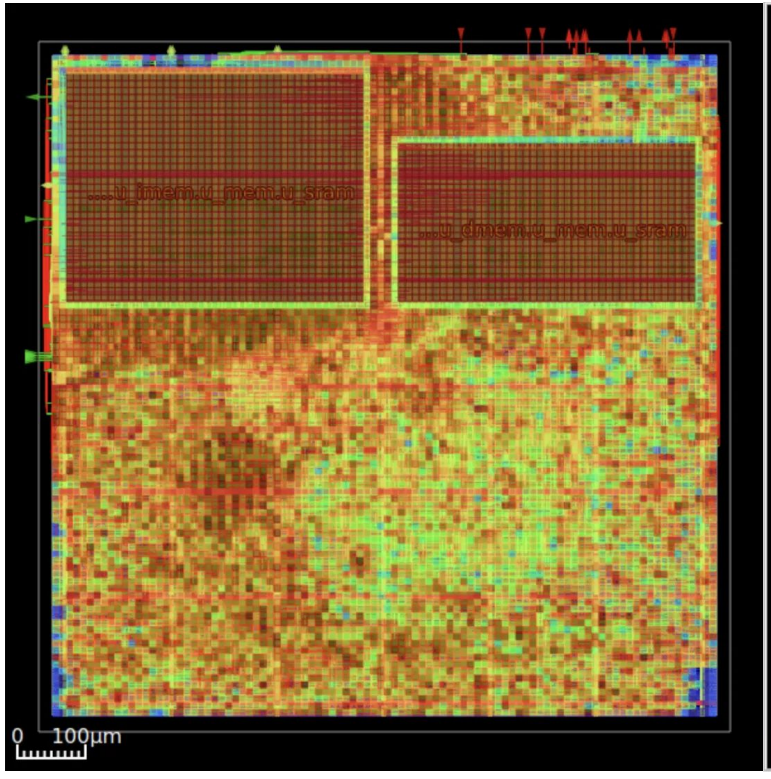
How full each region is (red = packed, blue = empty). 72.6% overall. Dense near the SRAMs where CPU/memory logic clusters, breathing room everywhere else — full enough to fit 1 mm², empty enough to still be routable.

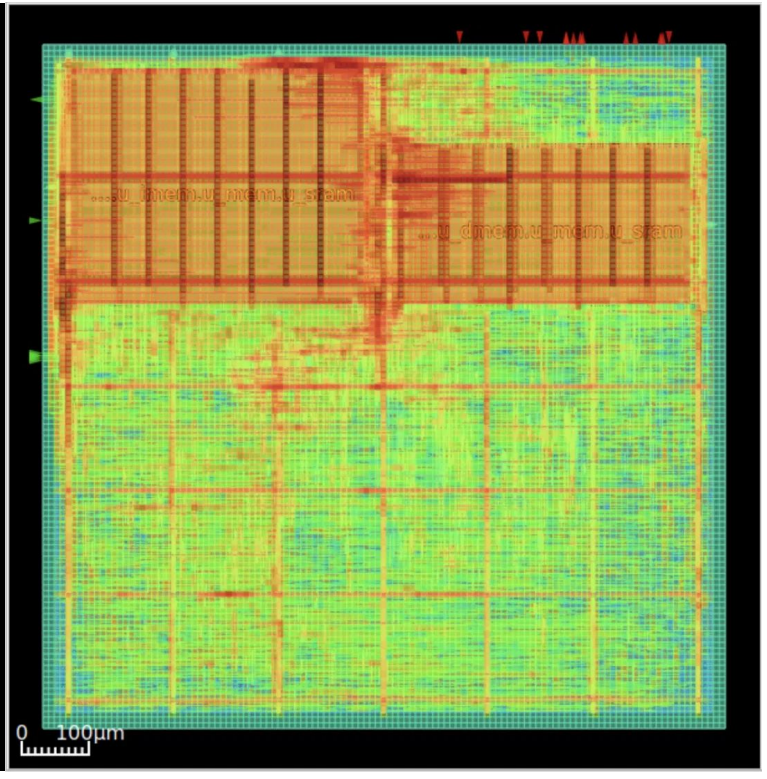
Routing Congestion

How close routing came to running out of wire tracks (red = nearly exhausted). The tight spots are exactly where expected — over and between the SRAMs, where all CPU↔memory traffic funnels. The router closed everything: 0 DRC violations

Why more dense near the macros?

Because the placer pulls connected cells close to shorten wires — and the fetch-gather, scatter units, bus adapters, and CPU all talk to the SRAMs constantly, so they got packed tight against the macros. Physics of the tool: heavily-connected logic clusters at what it talks to.



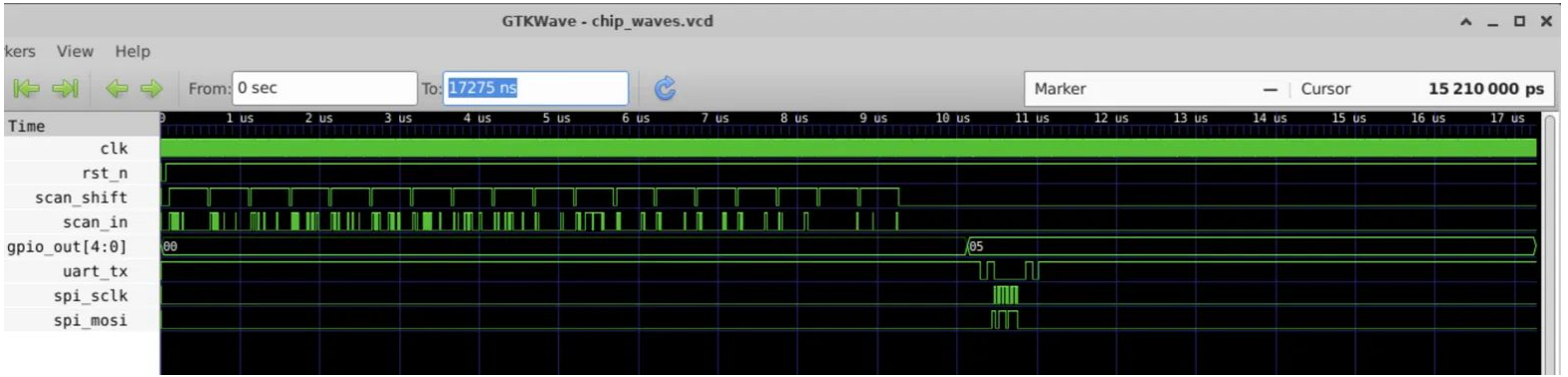


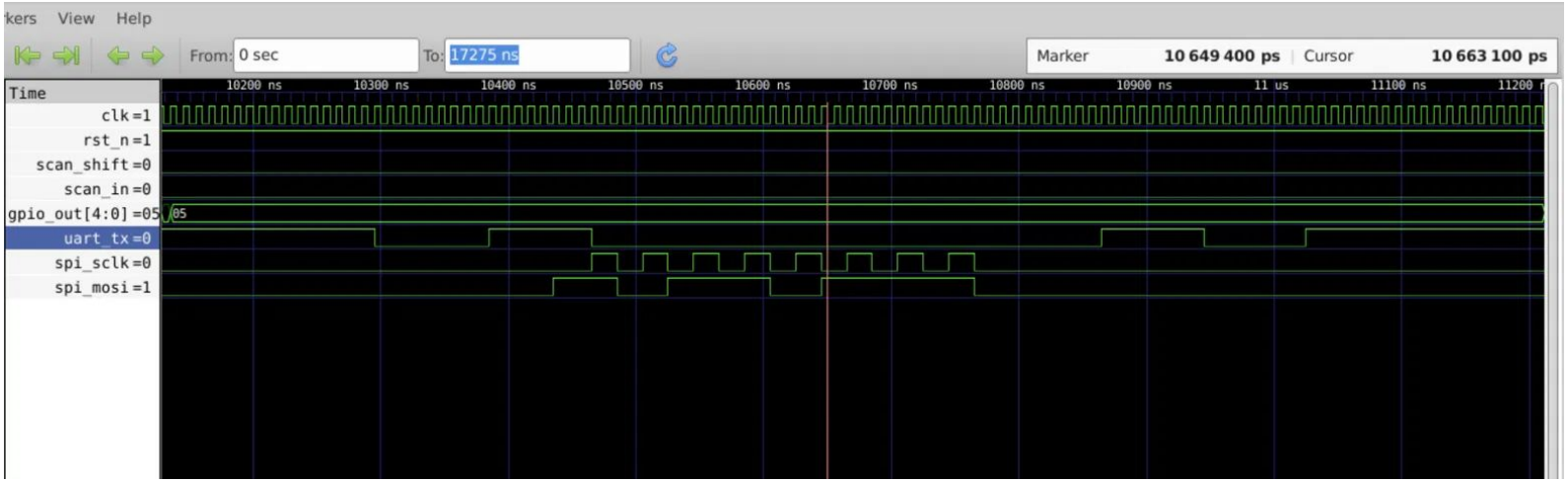
Extra Slide: Verification: scan-load → boot → GPIO/UART/SPI

What this verifies: this is our front-end verification — RTL simulation, done before physical design — shown here because the review asks whether the layout implements the *intended* design. The proof is a chain: this simulation proves the RTL logically works end-to-end — program scan-loaded (the only way to get code into the real chip), CPU boots, all three peripherals produce exactly the expected values — and then LVS proves the layout implements that same netlist device-for-device. Front-end proves it's the right circuit; back-end proves it was built right.

```
/foss/designs/ibex_soc > P=/foss/pdks/ciel/gf180mcu/versions/7b70722e33c03fcb5da
bcf4d479fb0822d9251c9/gf180mcuD/libs.ref
iverilog -g2012 -o /tmp/sim_rtl -s tb_chip_v2 tb/tb_chip_v2.sv rtl/*.sv rtl/pico
rv32.v $P/gf180mcu_fd_ip_sram/verilog/gf180mcu_fd_ip_sram__sram256x8m8wm1.v $P/g
f180mcu_fd_ip_sram/verilog/gf180mcu_fd_ip_sram__sram64x8m8wm1.v 2>&1 | grep -c e
rror ; vvp /tmp/sim_rtl | tail -8
0
FSM set to RUN. CPU running...
------------------------------------------------
  GPIO out : 0x05 (expect 0x05)
  UART sent: 0x41 (expect 0x41)
  SPI MOSI : 0xb7 (expect 0xB7), bits=8
------------------------------------------------
PASS: SoC boots via scan-configured FSM!
tb/tb_chip_v2.sv:100: $finish called at 17275000 (1ps)
```

Extra Slide: Verification: scan-load → boot → GPIO/UART/SPI





RTL simulation of the whole chip, doing what real silicon will do. Top: the program is loaded bit-by-bit through the scan chain (the bursts, 0–9 µs), then the CPU boots and drives all three peripherals — GPIO shows 05, UART sends a byte, SPI bursts. Bottom: zoomed in, the actual values are readable on the wires — UART = 0x41 ('A'), SPI = 0xB7. Exactly what the program was written to output.

Why it's here: the rubric asks whether the layout implements the *intended* design. This sim proves the design is correct; LVS proves the layout matches it exactly. Right circuit, built right

Extra Slide (more challenges): Hold Buffer Cap + Antenna Threshold

Problem 1 — hold fixing overflowed placement. Default flow inserted **3,055 hold buffers (~165k µm²)** to pad short paths, overflowing detailed placement (DPL) at our density. Fix: cap hold-fix buffer budget at **20%** (PL/GRT_RESIZER_HOLD_MAX_BUFFER_PCT) with **0.1 ns hold margin**, plus **cell padding 1-2 sites** so the placer keeps breathing room. Result: placement closes, hold still positive at every corner (worst +0.328 ns).

Problem 2 — antenna fixing overflowed placement. Blanket diode insertion (threshold 60) flooded the die with diodes → same overflow. Fix: **targeted heuristic insertion** — threshold **200**, diodes only on input ports (DIODE_ON_PORTS=in). Result: **0 antenna violations** with a fraction of the diodes.

Takeaway: both fixes are the same lesson — at 72.6% utilization, fix-up cells must be budgeted, not unlimited.